Василенко Е.Р. 090303-ПИа-о24

# Лабораторная работа №20
# Контейнеризация, CI/CD и эксплуатация Python-приложения

## 1. Цель работы

Получить практические навыки подготовки Python-приложения к развёртыванию и эксплуатации: освоить контейнеризацию приложения с помощью Docker и Docker Compose, создание Dockerfile, настройку переменных окружения и конфигурации, подключение базы данных в отдельном контейнере, сохранение данных через volume, реализацию health check, логирование, написание автоматических тестов и настройку CI/CD-пайплайна (GitHub Actions: lint, pytest, сборка Docker-образа).

## 2. Краткие теоретические сведения

Контейнеризация позволяет запускать приложение вместе с необходимым окружением в изолированном контейнере: код, зависимости, библиотеки и конфигурация упаковываются в единый образ, который одинаково работает на любой машине. Docker image — шаблон, на основании которого создаётся контейнер; контейнер — запущенный экземпляр image. Dockerfile описывает сборку образа основными инструкциями: FROM (базовый образ), WORKDIR (рабочая директория), COPY (копирование файлов), RUN (команда при сборке), ENV (переменные окружения), EXPOSE (документирование порта), CMD/ ENTRYPOINT (команда запуска).

Docker Compose описывает систему из нескольких компонентов декларативно в docker-compose.yml: для каждого сервиса задаются image или build, environment, ports, volumes, healthcheck и зависимости. Через volume (например, postgres_data) данные базы хранятся вне контейнера, поэтому docker compose down не удаляет данные. Сервисы обращаются друг к другу по имени сервиса (http://db:5432), а не по localhost.

Конфигурация приложения не должна жёстко прописываться в исходном коде — она задаётся переменными окружения (os.getenv). Файл .env для локальной разработки содержит реальные секреты и не должен попадать в Git;

в репозитории создаётся шаблон .env.example. Для распределённых приложений важен health check (GET /health), используемый Docker Compose для проверки состояния контейнера. Приложение должно логировать с помощью модуля logging (уровни DEBUG/INFO/WARNING/ERROR/CRITICAL), а не print().

CI/CD — совокупность практик автоматизации разработки и поставки ПО. CI (Continuous Integration) автоматически проверяет изменения после добавления в репозиторий: установка зависимостей, статические проверки, запуск тестов, сборка Docker-образа. В качестве CI-системы используется GitHub Actions — workflow размещается в .github/workflows/ci.yml и запускается при push и pull_request. Линтер Ruff выполняет автоматическую проверку качества кода. Для уменьшения размера production-образа применяется multi-stage сборка (builder image → production image).

**3. Индивидуальное задание**

Вариант 5 — «Ресторан». Контейнеризировать приложение «Ресторан» из предыдущих лабораторных работ: создать приложение и базу данных в Docker Compose, добавить автоматическую проверку API. Система должна состоять минимум из одного Python-приложения и базы данных в отдельном Docker-контейнере. Требуется: Dockerfile, .dockerignore, docker-compose.yml (приложение + PostgreSQL, volume для сохранения данных, healthcheck), переменные окружения, /health, логирование, автоматические тесты, CI pipeline (checkout → python setup → install → lint → pytest → docker build → docker compose config).

**4. Описание предметной области и структура проекта**

Предметная область — ресторан. Приложение предоставляет REST API каталога блюд: список блюд, получение блюда по ID, создание, удаление, а также эндпоинт /health для проверки работоспособности. Блюдо описывается полями id, name (название) и price (цена). Каталог при старте заполняется пятью стартовыми блюдами (seed), если база пуста. Хранение — PostgreSQL в отдельном контейнере; для локальных тестов используется SQLite через переменную окружения DATABASE_URL.

```
lab_20/
├── app/
│   ├── __init__.py
│   ├── main.py             # Flask API, /health, логирование, seed
│   └── database.py         # модель Dish, init_db
├── tests/test_api.py       # 12 тестов API (health, CRUD, валидация)
├── .github/workflows/ci.yml   # CI: lint + pytest + docker build
├── Dockerfile              # образ python:3.12-slim + HEALTHCHECK
├── .dockerignore
├── docker-compose.yml      # app + postgres, volume, healthcheck
├── .env.example
├── .gitignore
├── requirements.txt
└── README.md               # документация по запуску
```

## 5. Архитектура приложения

Архитектура приведена на рисунке 20.1: клиент обращается к Flask-приложению (контейнер app, порт 8000), которое хранит данные в PostgreSQL (контейнер db), подключение задано переменной окружения DATABASE_URL. Данные БД сохраняются в именованном volume postgres_data. Работоспособность app и db проверяется Docker HEALTHCHECK, а запуск с зависимостью от готовности БД задан через depends_on с condition: service_healthy.

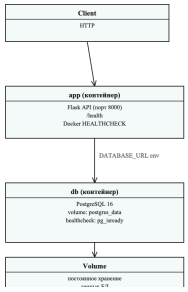



Рисунок 20.1 – Архитектура системы «Ресторан» в Docker Compose

| Компонент | Образ | Порт | Ответственность |
|---|---|---|---|
| app | build: . (python:3.12-slim) | 8000 | Flask API, /health, логирование |
| db | postgres:16 | 5432 | PostgreSQL, данные каталога |
| постgres_data | volume | — | постоянное хранение данных БД |

```
$ docker compose up --build   # сборка и запуск системы
$ curl http://localhost:8000/health   # проверка работоспособности
```

```
{"service": "restaurant", "status": "healthy"}
$ docker compose down         # остановка; данные остаются в volume
```

## 6. Реализация приложения

Приложение построено на Flask и Flask-SQLAlchemy через фабрику create_app: подключение к БД берётся из переменной окружения DATABASE_URL (в Compose — postgresql+psycopg://app:app_password@db:5432/restaurant). Каждый запрос логируется: метод, путь, статус и время обработки. Эндпоинты /dishes и /api/dishes дублируются как алиасы для удобства интеграций.

Файл: app/main.py — фабрика приложения, /health и логирование

```
def create_app() -> Flask:
    app = Flask(__name__)
    app.config["SQLALCHEMY_DATABASE_URI"] = os.getenv(
        "DATABASE_URL",
        "postgresql+psycopg://app:password@db:5432/restaurant",
    )
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

    db.init_app(app)
    with app.app_context():
        init_db()

    @app.before_request
    def log_request():
        request.environ["_start_time"] = time.perf_counter()

    @app.after_request
    def log_response(response):
        duration = (time.perf_counter() - request.environ.get("_start_time",
time.perf_counter())) * 1000
        logger.info("%s %s -> %s (%.1f ms)", request.method, request.path,
                    response.status_code, duration)
        return response

    @app.get("/health")
    def health():
        return jsonify({"status": "healthy", "service": "restaurant"})

    @app.get("/")
    def root():
        return jsonify({
            "service": "Restaurant API",
            "endpoints": ["/health", "/dishes", "/api/dishes"],
```

Файл: app/main.py — CRUD блюд

```
    @app.get("/dishes")
    @app.get("/api/dishes")
    def list_dishes():
        dishes = Dish.query.order_by(Dish.name).all()
        return jsonify([d.to_dict() for d in dishes])

    @app.get("/dishes/<int:dish_id>")
    @app.get("/api/dishes/<int:dish_id>")
    def get_dish(dish_id: int):
        dish = db.session.get(Dish, dish_id)
        if dish is None:
            return jsonify({"error": "Блюдо не найдено"}), 404
        return jsonify(dish.to_dict())

    @app.post("/dishes")
```

```
    @app.post("/api/dishes")
    def create_dish():
        data = request.get_json(silent=True) or {}
        name = (data.get("name") or "").strip()
        try:
            price = float(data.get("price", 0))
        except (TypeError, ValueError):
            return jsonify({"error": "Цена должна быть числом"}), 400
        if not name:
            return jsonify({"error": "Название блюда не может быть пустым"}),
400
        if price <= 0:
            return jsonify({"error": "Цена должна быть положительной"}), 400
        dish = Dish(name=name, price=price)
        db.session.add(dish)
        db.session.commit()
        logger.info("Создано блюдо %s (ID %s)", dish.name, dish.id)
        return jsonify(dish.to_dict()), 201

    @app.delete("/dishes/<int:dish_id>")
    @app.delete("/api/dishes/<int:dish_id>")
    def delete_dish(dish_id: int):
        dish = db.session.get(Dish, dish_id)
        if dish is None:
            return jsonify({"error": "Блюдо не найдено"}), 404
        db.session.delete(dish)
        db.session.commit()
        logger.info("Удалено блюдо ID %s", dish_id)
        return "", 204

    return app
```

Файл: app/database.py — модель Dish и инициализация

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()


class Dish(db.Model):
    """Блюдо ресторана."""

    __tablename__ = "dishes"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(120), nullable=False)
    price = db.Column(db.Float, nullable=False)

    def to_dict(self) -> dict:
        return {"id": self.id, "name": self.name, "price": self.price}


def init_db() -> None:
    db.create_all()
    if Dish.query.count() == 0:
        db.session.add_all([
            Dish(name="Борщ", price=350.0),
            Dish(name="Цезарь", price=420.0),
            Dish(name="Стейк", price=890.0),
            Dish(name="Паста", price=450.0),
            Dish(name="Чизкейк", price=320.0),
        ])
        db.session.commit()
```

## 7. Dockerfile и .dockerignore

Образ собирается на официальном python:3.12-slim: устанавливаются зависимости из requirements.txt (слой кэшируется, пока не изменился файл), копируется исходный код, задаётся переменная PORT=8000, документируется

порт EXPOSE и через HEALTHCHECK проверяется /health при помощи python-скрипта urllib (в slim-образе curl отсутствует). Запуск выполняется корректной командой gunicorn (production WSGI-сервер).

Файл: Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

ENV PORT=8000
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://
localhost:8000/health')"

CMD ["python", "-m", "gunicorn", "--bind", "0.0.0.0:8000", "app.main:app"]
```

Файл: .dockerignore — исключение служебных файлов из образа

```
.git
.venv
__pycache__
.pytest_cache
*.pyc
.env
.idea
*.db
```

| Инструкция | Назначение |
|---|---|
| FROM python:3.12-slim | официальный базовый образ |
| WORKDIR /app | рабочая директория |
| COPY requirements.txt . / RUN pip install | установка зависимостей |
| COPY . . | копирование исходного кода |
| ENV / EXPOSE | переменная окружения и порт |
| HEALTHCHECK | проверка /health |
| CMD gunicorn | корректная команда запуска |

## 8. Docker Compose

docker-compose.yml описывает два сервиса. db — контейнер postgres:16: имя БД, пользователь и пароль задаются переменными окружения (не в коде), данные сохраняются в volume postgres_data (монтируется в /var/lib/postgresql/data), доступность проверяется healthcheck pg_isready. app собирается из Dockerfile, пробрасывает порт 8000, получает DATABASE_URL через environment и запускается только после готовности БД (depends_on condition:

service_healthy). Внутри сети Docker сервисы обращаются друг к другу по имени (db:5432).

Файл: docker-compose.yml

```
services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql+psycopg://app:app_password@db:5432/restaurant
      - PORT=8000
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 30s
      timeout: 5s
      retries: 3

  db:
    image: postgres:16
    environment:
      - POSTGRES_DB=restaurant
      - POSTGRES_USER=app
      - POSTGRES_PASSWORD=app_password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d restaurant"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:
```

Файл: .env.example — шаблон переменных окружения

```
DATABASE_URL=postgresql+psycopg://app:app_password@db:5432/restaurant
PORT=8000
```

Файл .env исключён из Git (.gitignore), реальные пароли в исходный код не попадают — они задаются через environment в docker-compose.yml или CI-Secrets.

**9. Health Check и логирование**

GET /health возвращает {"status": "healthy"}. Он используется и Docker HEALTHCHECK, и docker-compose healthcheck — Docker считает контейнер healthy, только когда запрос завершается успешно. Все входящие запросы логируются модулем logging с уровнями: после каждого ответа фиксируются метод, путь, HTTP-статус и время ответа в миллисекундах; создание и удаление

блюда дополнительно пишутся информационными сообщениями. Пример журнала приведён на рисунке 20.2.




Рисунок 20.2 – Журнал запросов приложения (logging)

## 10. Тестирование

Автоматические тесты (pytest) используют TestClient Flask и временную SQLite-базу: перед импортом приложения переопределяется DATABASE_URL. Покрыты health-check, корневой эндпоинт, список блюд (5 стартовых), алиас /api/dishes, получение блюда и ошибка 404, создание блюда с кодом 201 и варианты невалидных данных (пустое имя, отрицательная цена, нечисловая цена — 400), удаление с кодом 204 и 404 при отсутствии. Тесты не требуют Docker и PostgreSQL — они выполняются на SQLite, что важно для CI.

Файл: tests/test_api.py

```
import os
import sys
import tempfile
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

import pytest

os.environ["DATABASE_URL"] = f"sqlite:///{tempfile.mkdtemp()}/test.db"

from app.main import app


@pytest.fixture()
def client():
    app.config.update(TESTING=True)
    with app.test_client() as test_client:
        yield test_client


class TestHealth:
    def test_health(self, client):
        response = client.get("/health")

        assert response.status_code == 200
        body = response.get_json()
        assert body["status"] == "healthy"
```

```
        assert body["service"] == "restaurant"

    def test_root(self, client):
        response = client.get("/")
        assert response.status_code == 200
        assert response.get_json()["service"] == "Restaurant API"


class TestDishesCrud:
    def test_list_dishes(self, client):
        response = client.get("/dishes")

        assert response.status_code == 200
        dishes = response.get_json()
        assert len(dishes) == 5
        assert any(d["name"] == "Борщ" for d in dishes)

    def test_api_alias_list(self, client):
        assert client.get("/api/dishes").status_code == 200

    def test_get_dish(self, client):
        response = client.get("/dishes/1")

        assert response.status_code == 200
        assert response.get_json()["id"] == 1

    def test_get_missing_dish(self, client):
        response = client.get("/dishes/999")

        assert response.status_code == 404
        assert "не найдено" in response.get_json()["error"]

    def test_create_dish(self, client):
        response = client.post("/dishes", json={"name": "Суп", "price": 250.0})

        assert response.status_code == 201
        assert response.get_json()["name"] == "Суп"
        assert client.get("/dishes/6").get_json()["price"] == 250.0

    def test_create_dish_invalid_name(self, client):
        response = client.post("/dishes", json={"name": "", "price": 250.0})

        assert response.status_code == 400
        assert "пустым" in response.get_json()["error"]

    def test_create_dish_invalid_price(self, client):
        response = client.post("/dishes", json={"name": "Суп", "price": -10})

        assert response.status_code == 400

    def test_create_dish_wrong_type(self, client):
        response = client.post("/dishes", json={"name": "Суп", "price":
"дорого"})

        assert response.status_code == 400

    def test_delete_dish(self, client):
        created = client.post("/dishes", json={"name": "Для удаления", "price":
100.0})
        dish_id = created.get_json()["id"]

        assert client.delete(f"/dishes/{dish_id}").status_code == 204
        assert client.get(f"/dishes/{dish_id}").status_code == 404

    def test_delete_missing_dish(self, client):
        assert client.delete("/dishes/999").status_code == 404
```

Результаты запуска: 12 тестов прошли успешно (12 passed), скорость ~0.3 с (рисунок 20.3). Статическая проверка качества кода ruff check app tests также проходит без замечаний.

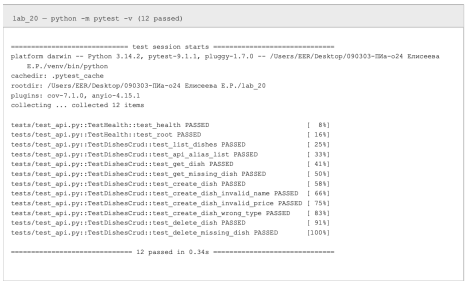



Рисунок 20.3 – Результаты pytest (12 passed) и ruff

## 11. CI/CD pipeline (GitHub Actions)

Workflow ci.yml запускается при push в main и при pull_request. Этапы: checkout кода, установка Python 3.12 (actions/setup-python), установка зависимостей, линтер ruff (CI завершается ошибкой при нарушениях), запуск тестов pytest, сборка Docker-образа docker build и проверка корректности конфигурации docker compose config. Если тесты или линтер не прошли, последующие этапы не выполняются — pipeline падает.

Файл: .github/workflows/ci.yml

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
```

```
        pip install --upgrade pip
        pip install -r requirements.txt

    - name: Lint (ruff)
      run: ruff check app tests

    - name: Run tests
      run: pytest -v

    - name: Docker build
      run: docker build -t restaurant-app .

    - name: Docker Compose config check
      run: docker compose config
```

| Этап | Команда | Провал pipeline |
|---|---|---|
| Checkout | actions/checkout@v4 | — |
| Python setup | actions/setup-python@v5 (3.12) | — |
| Install | pip install -r requirements.txt | — |
| Lint | ruff check app tests | да |
| Tests | pytest -v | да |
| Docker build | docker build -t restaurant-app . | да |
| Compose check | docker compose config | да |

## 12. Демонстрация работы

Живая демонстрация API (Flask-сервер на SQLite): health check, список блюд, создание блюда (201), получение по ID, удаление (204), ошибки валидации (400) и 404 для отсутствующего блюда (рисунок 20.4).




Рисунок 20.4 – REST API «Ресторан»: health, CRUD, валидация, 404

## 13. Вывод

В ходе лабораторной работы выполнена контейнеризация Python-приложения «Ресторан»: созданы Dockerfile на официальном образе python:3.12-slim с HEALTHCHECK и корректной командой запуска (gunicorn), .dockerignore и docker-compose.yml, в котором приложение и база данных PostgreSQL 16 работают в отдельных контейнерах, подключение задано

переменной окружения DATABASE_URL, данные сохраняются в volume postgres_data (переживают docker compose down), а запуск приложения ожидает готовности БД через healthcheck.

Конфигурация вынесена из кода в переменные окружения, пароль БД не хранится в исходном коде, в репозиторий включён шаблон .env.example, а .env исключён через .gitignore. Реализованы health check (/health) и логирование запросов модулем logging. Написаны 12 автоматических тестов (pytest), покрывающих health, CRUD и валидацию; тесты выполняются на SQLite и не требуют Docker. Настроен CI/CD-пайплайн GitHub Actions: checkout → python setup → install → lint (ruff) → pytest → docker build → docker compose config, причём pipeline останавливается при любой ошибке на ранних этапах. Полученные навыки контейнеризации, автоматизации проверок и эксплуатации приложений применимы к развёртыванию любого Python-проекта.